Sequential vs Parallel Processing

26 July 2025 21:56

Pro-tip*

Sequential Processing

- tasks which are executed one by one or say one after the other
- basically each task starts only after the previous one has completed

Ex: Common in single-core CPUs or when task dependency is critical

Parallel Processing

- In parallel processing, multiple tasks are executed simultaneously by distributing the workload across multiple CPUs cores, GPUs or any other resources.
- Basically tasks are divided and assigned to different processing units to run concurrently.

1

Sequential Processing

How many tickets can be booked at once

3

Feature	Sequential Processing	Parallel Processing
Execution Style	Step-by-step (one at a time)	Simultaneous execution
Resource Usage	Single core/thread	Multiple cores/threads
Speed	Slower for large tasks	Faster for large/divisible tasks
Complexity	Easier to implement	More complex (sync, data sharing, etc.)
Task Dependency	Suitable for dependent tasks	Best for independent tasks
Scalability	Limited	Highly scalable
Example	Simple loop iteration	Matrix multiplication, image processing

(for loop)

Nokia 1600 — Simple

iPhone 15 Pro max — Complex

1 year

30K upwards

if display is gone..

2nd year

DO IT AT YOUR OWN RISK

Press **windows + R**-->
write **cmd**-->
and copy paste the below command:
wmic cpu get NumberOfCores,NumberOfLogicalProcessors

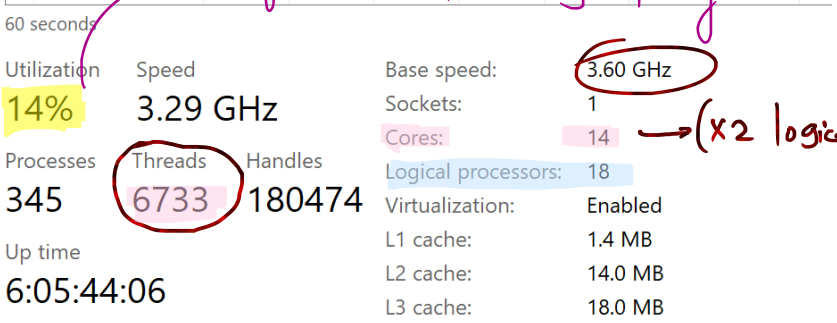
```
C:\Users\thesi>wmic cpu get NumberOfCores,NumberOfLogicalProcessors
NumberOfCores  NumberOfLogicalProcessors
14             18
```

Core 2 Duo vs Dual Core

two cores but a terminology given by Intel

any general cpu having 2 cores is 'dual core'

'out of available processing capacity'



→ (x2 logical processors)

core vs thread

core → is a physical processing unit within the CPU (central processing unit) and each core can independently execute instructions and run tasks.

Thread → is a virtual processing unit managed by CPU and each core can run one or more threads.

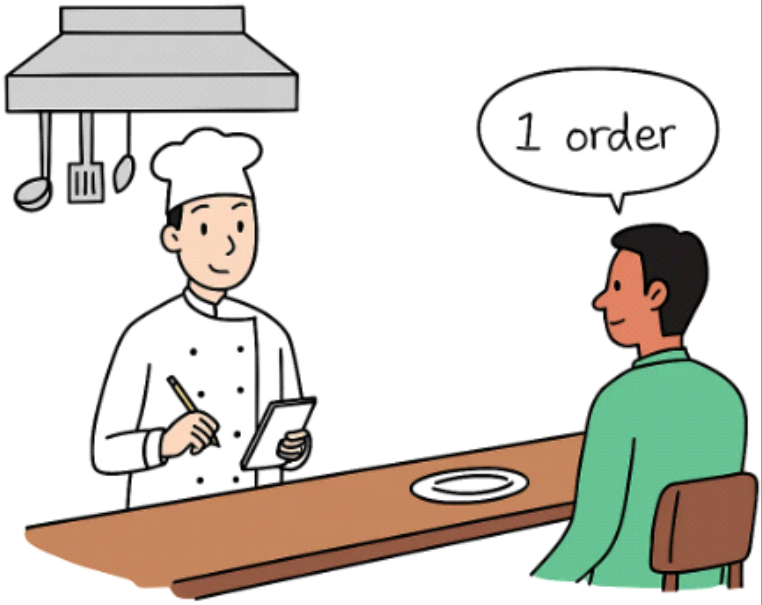
Hardware thread → Logical Processor

Software thread → managed by OS scheduler.

Visual analogy

Imagine your CPU as a restaurant:

- cores : Chefs – 8
- logical processors : cooking stations/chef
  (Each chef can work on two meals at once → 16 logical processors
- → Software threads: orders ←
  Each order handled by a station

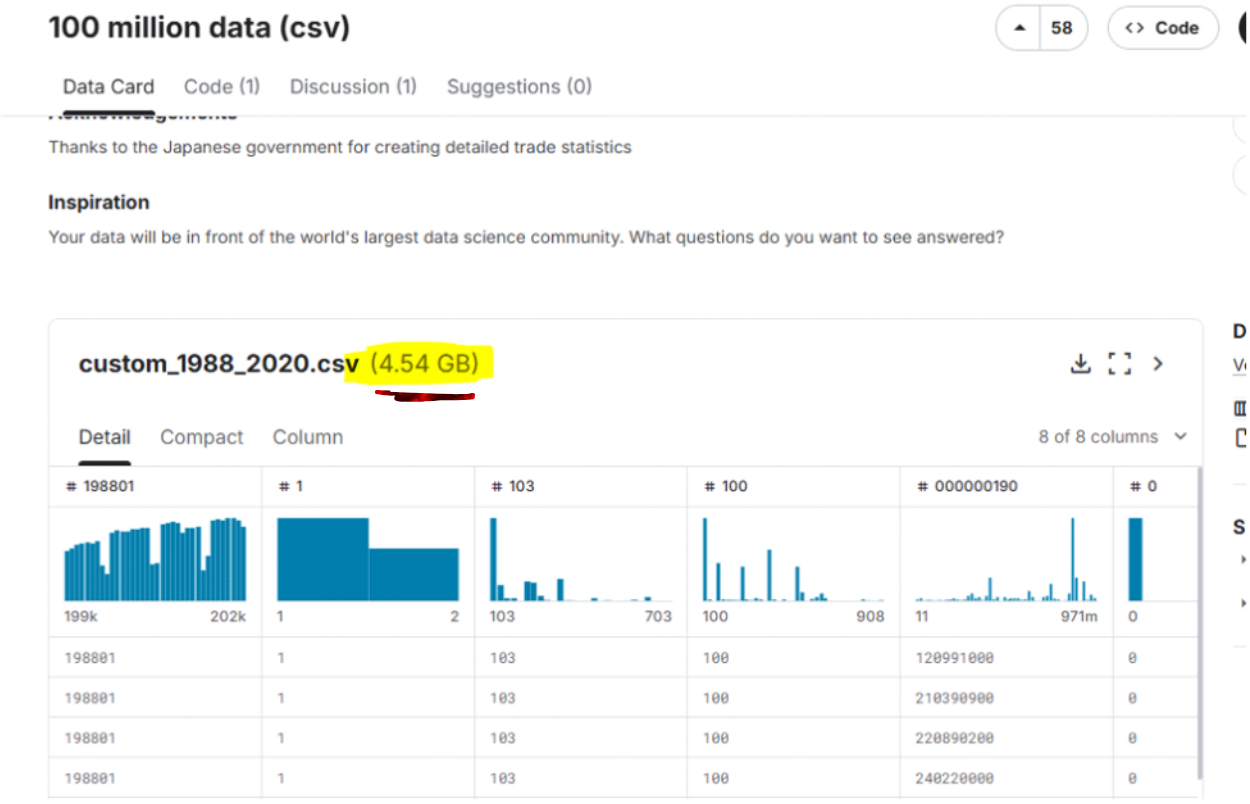


Note: Most modern CPUs support simultaneous Multi-Threading (SMT) or hyper-Threading (HT)

Task # Let us say if you are working with large datasets (~100M rows) on a 16GB RAM or lesser RAM and 256GB SSD

↓

Can the machine handle this ??



https://www.kaggle.com/datasets/zanjibar/100-million-data-csv